

Claude Code Skills

Technical Deep-Dive

Progressive Disclosure, Automatic Invocation, and Compositional Architecture

Generated: 2025-11-05
Sources: Sean Kochel & IndyDevDan

Executive Summary

Claude Code Skills are agent-invoked, context-efficient features for packaging custom expertise into reusable solutions. They represent the highest-level compositional unit in Claude Code architecture, rated 8/10 for their ability to standardize workflow patterns through progressive disclosure and automatic invocation.

Feature	Skills	MCPs	Sub-agents	Slash Commands
Agent-Invoked	✓ YES	✗ NO	✓ YES	✗ NO (Manual)
Context Efficient	✓ Progressive	✗ Explodes	✓ Isolated	✓ Direct
Context Persistence	✓ Main	✓ Main	✗ Lost	✓ Main
Modularity	HIGH	Medium	Medium	Low
Composition	✓ All below	✗ No skills	✓ Commands	✓ Skills

Core Quote (Sean Kochel): "Skills are arguably a bigger deal than MCPs because they encode systematic methodologies, not just tool access—they teach Claude to work like experienced professionals."

1. Progressive Disclosure: How Skills Load Context Incrementally

Progressive disclosure is the killer feature that makes skills context-efficient. Unlike MCP servers which 'explode your context window on bootup,' skills load resources incrementally only when needed.

The Three-Level Loading Mechanism

Skills use a lazy-loading architecture with three distinct levels:

Level	What Loads	When	Token Cost	Purpose
1. Metadata	skill.json - Name - Description - Keywords	Always (on startup)	~50 tokens	Agent decides if relevant
2. Instructions	skill.md - How to use - When to use - Defaults	When agent detects match	~500 tokens	Agent reads workflow
3. Resources	resources/ - Files - Templates - Examples - Skill Variables	When needed (e.g., on demand)	Variable (1K-10K)	Agent pulls what's needed

Level 1: Metadata Discovery

On Claude Code startup, all skill metadata loads into the agent's awareness. This is lightweight (~50 tokens per skill) and tells the agent WHAT skills exist and WHEN to consider using them.

```
{
  "name": "UI Guidelines Skill",
  "description": "Ensures brand consistency for React components",
  "keywords": ["component", "UI", "brand", "design system"],
  "triggers": ["building component", "creating UI", "design consistency"]
}
```

Level 2: Instructions Loading

When the agent detects a task matching skill keywords (e.g., user says 'create a button component'), it loads skill.md (~500 tokens). This provides the HOW—the systematic workflow the agent should follow.

```
# UI Guidelines Skill

## When to Use
Automatically invoke when building React components.

## Workflow
1. Check design system for existing patterns
2. Load brand colors from resources/colors.json
3. Apply typography from resources/typography.json
4. Validate against component checklist

## Resources
- resources/design-system.pdf
- resources/brand-colors.json
- resources/component-checklist.md
```

Level 3: Resource Pulling

Only if executing the skill does the agent pull actual resource files (1K-10K tokens). These might be design docs, code templates, configuration files—whatever the skill needs to complete its work.

Critical Comparison: MCP servers load ALL tools and schemas on startup (explosion). Skills load metadata → instructions → resources progressively (precision).

2. Skill Creator: The Meta-Skill (Double Example)

Skill Creator demonstrates skills in two ways: (1) HOW it works as a skill itself, and (2) HOW it creates other skills. This 'meta-skill' pattern shows the full power of skills architecture.

Example A: Skill Creator AS a Skill

Skill Creator is itself a skill with progressive disclosure:

```
User: "Create a skill for enforcing UI guidelines"

→ Agent sees 'create a skill' in metadata keywords
→ Agent loads Skill Creator instructions
→ Agent asks targeted questions:
- What component types? (buttons, forms, modals)
- Where are guidelines? (.claude/system/design-system.md)
- When to invoke? (when building React components)
- What resources needed? (colors.json, typography.json)
→ Agent generates skill.json + skill.md + resources/
→ Agent packages as zip file
→ User downloads and installs to .claude/skills/
```

Example B: What Skill Creator CREATES

The UI Guidelines skill created by Skill Creator becomes:

```
.claude/skills/ui-guidelines/
■■■ skill.json (metadata - 50 tokens)
■ {
■   "name": "UI Guidelines",
■   "keywords": ["component", "button", "form", "UI"],
■   "triggers": ["create component", "build UI"]
■ }
■■■ skill.md (instructions - 500 tokens)
■ # Apply brand standards to components
■ ## Workflow
■ 1. Load colors from resources/colors.json
■ 2. Apply typography from resources/typography.json
■ 3. Validate component against checklist
■■■ resources/
■ ■■■ colors.json (200 tokens)
■ ■■■ typography.json (150 tokens)
■ ■■■ checklist.md (300 tokens)
```

The Meta Pattern: Skill Creator is a skill that creates skills. This is 'build the thing that builds the thing'—a core agentic abstraction pattern.

3. Composition Hierarchy: Skills at the Top

Skills represent the highest compositional level in Claude Code architecture. Understanding this hierarchy is critical for knowing when to use which feature.

Level	Feature	Composes	Invoked By
TOP	Skills	MCPs + Sub-agents + Commands	Agent (automatic)
MIDDLE	MCP Servers	External APIs	Agent + Skills
MIDDLE	Sub-agents	Commands	Agent + Skills
MIDDLE	Slash Commands	Skills (recursive)	User (manual)
BOTTOM	Prompts	N/A (primitive)	Everything

Key Rule (IndyDevDan): "Custom Slash Commands are the primitive of agentic coding. Master prompts before mastering compositional features. Everything is a prompt in the end (tokens in, tokens out)."

4. When to Use Skills vs Alternatives

The Decision Framework

- **One-off task?** → Use Slash Command (don't over-engineer)
- **Managing problem DOMAIN?** → Use Skill (create, remove, list, merge operations)
- **External integration?** → Use MCP Server (Jira, databases, APIs)
- **Parallel isolated work?** → Use Sub-agent (context protection)
- **Automatic behavior?** → Use Skill (agent-invoked)
- **Manual trigger?** → Use Slash Command (user control)

Final Rating & Verdict

Skills: **8/10** (IndyDevDan rating)

Pros: Agent-invoked, context-efficient progressive disclosure, dedicated file system structure, composes all features below, higher compositional abstraction

Cons: Doesn't go all the way (can't nest sub-agents, can't nest prompts in /commands directory within skill), reliability concerns when chaining multiple skills

Bottom Line: Skills are canonized prompt engineering + modularity. Not revolutionary in capability, but valuable for standardization and shareability. Use for repeatable agent-invoked domain management, not one-off tasks.